



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.walls  
self.food_positions  
self.opponents  
self.agent_pos
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent environment. `self.opponents` is a list of independently-moving entities, and their behavior directly affects the primary agent's performance measure (collision penalty). Because other entities are acting and their actions matter to the agent's outcome, the environment qualifies as multi-agent

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything an agent has perceived so far in order. An agent's choice of action at any point can depend on the entire percept sequence observed up to that point, not just the current percept.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

This method represents the Sensors (S).

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable. The dictionary returned includes `agent_pos`, `opponent_positions`, `smells_food`, `hit_wall`, `collision`, `score`, and `remaining_food` — but it does not expose the full grid layout (`self.walls` is never revealed in full, only whether the agent's own current cell is a wall) and does not reveal the location of remaining food, only a boolean at the agent's current cell and a count. The agent can't reconstruct the complete world state from a single percept, so its sensors give it an incomplete snapshot rather than full access to the true environment state

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure (P) is being updated.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Performance measures must be defined in terms of what is actually desirable in the environment (external, observable outcomes) rather than agent's internal computation. If performance were measured by internal processing effort (e.g., CPU cycles or lines of code executed), an agent could maximize its score by running infinite useless loops while standing still. A true performance measure evaluates what the agent achieves in its environment, not how hard it thinks.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If `self.toxic_traps` exists in the environment state but is not exposed through `get_percept()`, it strictly worsens the environment's observability — the environment becomes more partially observable than before. The agent now has a portion of the true state (hazard locations) that it has no sensory access to at all, meaning it cannot even in principle reconstruct the complete state from its percept history. This also has a knock-on effect on rational decision-making: the agent must act under greater uncertainty, potentially needing internal state/memory or probabilistic reasoning to compensate.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality means selecting the action expected to maximize performance measure given the percept sequence and built-in knowledge. Without `smells_toxin`, the agent has no way to distinguish a safe cell from a lethal one and can only discover danger after incurring the -15 penalty. By adding this boolean sensor, the agent gains the information needed to make an informed choice (e.g.: avoid moving into a cell it currently occupies that smells of toxin or avoid stepping toward one). This directly improves rationality because the agent can now select actions that better maximize expected score, using accurate, timely percepts rather than acting blindly.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic example from Lecture 01 (Russell & Norvig's vacuum-cleaner world) is the case where the performance measure rewards the agent for cleaning dirt but doesn't penalize creating new dirt. A poorly designed rational agent exploiting this metric could clean a square, then dirty it again, then clean it again repeatedly farming reward from the same square forever, technically maximizing the (flawed) performance measure while never actually achieving the real goal of a clean floor. It illustrates why performance measures must be tied to the actual desired outcome in the world rather than a proxy metric that can be gamed.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING